

Date:

BONAFIDE CERTIFICATE

This is to certify that **Gokulakrishnan V**, a student of **B.E Computer Science and Engineering** at **Kathir College of Engineering**, has successfully completed an internship at **Cybervault Innovation and Technologies Pte. Ltd.** from **July 2024** to **September 2024**.

During this period, the student worked on the project titled "**Cybervault Password Manager**" Under the supervision of **Mrs. Ramadevi S**, and has fulfilled all the requirements as per the internship program guidelines.

We certify that the work done by the student is genuine and reflects their efforts during the tenure of the internship.

We wish them success in their future endeavours.

Authorized Signatory
Ramadevi S
Data Analyst
Cybervault Innovations and
Technologies Ptd Ltd.

Authorized Signatory
SubaShree
HR Manager
Cybervault Innovations
and Technologies Ptd Ltd.

Website: www.cybervaultit.com

Password Manager

OBJECTIVE:

This project aims to create a secure and easy-to-use password manager that helps people safely store and manage their passwords. By using strong encryption and secure login methods, the system will ensure that all stored passwords are protected. It will also allow users to easily check the strength of their passwords and generate strong ones when needed.

INTRODUCTION:

1. TECHNICAL STACK

1. Flask (Python):

- Flask is a lightweight web framework that enables rapid backend development in Python, facilitating the creation of RESTful APIs.
- Its modular architecture allows easy integration with databases and authentication systems.
- This flexibility makes it a preferred choice for web application development.

2. JavaScript:

- JavaScript enhances frontend interactions and allows for dynamic functionality, improving user engagement.
- It enables real-time updates and form validations, working seamlessly with HTML and CSS for a complete user experience.
- Mastery of JavaScript is essential for creating interactive web applications.

3. HTML/CSS:

- HTML structures web pages, while CSS styles and formats the content, ensuring visual appeal.
- Together, they form the foundation for designing user interfaces, employing responsive design techniques for various devices.
- Proficiency in HTML and CSS is crucial for any web developer.

2. SECURITY KNOWLEDGE

AES and ECC Encryption Algorithms:

- AES is a symmetric encryption standard for securing sensitive data, while ECC is an asymmetric method for secure key exchange.
- Using these algorithms ensures user data confidentiality and protection against unauthorized access.
- Knowledge of these encryption techniques is vital for developing secure applications.

3. PYTHON LIBRARIES

1. Pandas:

- Pandas is a powerful library for data manipulation, providing DataFrames for efficient cleaning and analysis of structured data.
- It simplifies complex data operations, making it invaluable for data analysis tasks.
- Proficiency in Pandas enhances data handling capabilities in applications.

2. Plotly and Matplotlib:

- These libraries are used for creating interactive and static visualizations in Python.
- Plotly facilitates dynamic, web-based plots, while Matplotlib is known for high-quality static charts.
- Both are essential for visualizing data, aiding in better insights and decision-making.

3. CSV Handling:

- CSV handling in Python allows for easy import and export of tabular data using the `csv` module or Pandas.
- This capability is essential for applications requiring data exchange or backup functionalities.
- Understanding CSV handling improves data interoperability.

4. SECURITY EXPERTISE

1. Cryptography:

- The Cryptography library is essential for implementing secure encryption methods like AES and ECC.
- It provides tools for encrypting data and managing digital signatures, ensuring data security in applications.
- Effective use of cryptography protects sensitive information from unauthorized access.

2. Argon2:

- Argon2 is a modern password hashing algorithm that is secure against brute-force attacks.
- It uses memory-hard functions, allowing for configurable computational effort to resist cracking attempts.
- Integrating Argon2 into applications ensures secure password management and protection.

3. Flask-Session:

- Flask-Session is an extension for managing user sessions in Flask applications, allowing server-side storage of session data.
- This enhances security and scalability, facilitating seamless user experiences.
- Understanding Flask-Session is important for effective user authentication management.

5. USER INPUT VALIDATION

1. Password Policy:

- The Password Policy library enforces strong password requirements, ensuring users create secure passwords.
- By defining rules for length and complexity, it helps reduce weak passwords and enhance overall security.
- Implementing a password policy is crucial for protecting user accounts.

2. Regex:

- Regular expressions (regex) validate user input, ensuring data integrity by checking patterns like valid email formats or secure passwords.
- This step is essential for preventing malicious input and maintaining data quality.
- Mastery of regex improves the reliability of user input handling.

6. EMAIL HANDLING

Smtplib and Email.mime:

- The `smtplib` module allows for sending emails via SMTP, while `Email.mime` helps create well-structured email messages.
- This functionality is important for user notifications, password resets, and communication within applications.
- Knowledge of email handling enhances user engagement.

7. LOGGING AND DEBUGGING

Logging:

- The logging module tracks application behavior and logs events or errors, helping in troubleshooting and monitoring performance.
- Implementing logging gathers insights into application usage and identifies areas for improvement.
- Effective logging is crucial for maintaining robust applications.

8. DATABASE CONNECTION

Mysql.connector:

- The `mysql.connector` module establishes connections between Python applications and MySQL databases, facilitating SQL query execution and data retrieval.
- Understanding this module is essential for integrating backend data storage into web applications.
- This connection is vital for data-driven functionality.

PREREQUISITES:

1. TECHNICAL STACK

- **Flask (Python)** for backend development.
- **JavaScript** for frontend interactions and logic.
- **HTML/CSS** for designing the user interface.
- **MySQL** as the database for storing user data.

2. SECURITY KNOWLEDGE

We use AES and ECC encryption algorithm to protect passwords.

3. PYTHON LIBRARIES

1. DATA HANDLING and VISUALIZATION

- **Pandas** for data manipulation.
- **Plotly** and **Matplotlib** for data visualization.
- **CSV** handling for importing/exporting data

2. SECURITY EXPERTISE

- **Cryptograph** for implementing AES and ECC algorithms.
- **Argon2** for password hashing and management.
- **Flask_session** for session management.

3. USER INPUT VALIDATION

- **Password Policy** to enforce password strength requirements.
- **Regex** for validating user input and passwords.

4. EMAIL HANDLING

- **Smtplib** and **Email.mime** for sending emails.

5. LOGGING and DEBUGGING

- **Logging** for tracking application behavior and troubleshooting

6. DATABASE CONNECTION

- **Mysql.connector** for connect python and MySQL.

PROCEDURE:

1. Flask Application Setup

- To set up a Flask application, initialize it with ``app = Flask(__name__)`` and set a secure secret key.

Set up logging by using

``logging.basicConfig(level=logging.DEBUG)`` and define the session storage by setting ``app.config['SESSION_TYPE']`` to `'filesystem'`.

- Integrate Flask-Session by calling ``Session(app)`` for server-side session management.
- Finally, import MySQL Connector and establish a database connection using ``mysql.connector.connect()`` with the necessary credentials.

2. Database Connection and Query Execution in Flask

- The ``get_db_connection()`` function establishes a connection to the MySQL database.
- The ``execute_query()`` function runs SQL queries, optionally accepting parameters for dynamic queries, committing changes if needed, and returning fetched results.
- If an error occurs during query execution, it logs the error and rolls back the transaction.
- The function ensures proper resource management by closing the cursor and database connection in the end.

3. Database and Table Creation for Password Manager

- Creates a **PASSWORD_MANAGER** database and several tables:
password for storing passwords, **payments** for payment details, **category** for categorizing items, **folder** for folder management, **user** for user data, **delete_items** for tracking deleted records.
- Each table is structured with relevant fields like email, strength, and created_time to manage and organize user information securely.

4. Password Strength Policy Configuration

- The ``PasswordPolicy`` object is created to enforce password requirements for user accounts.
It specifies that passwords must be at least 8 characters long, include at least one uppercase letter, one digit, and one special character.
- This helps ensure that users create strong passwords. Implementing this policy enhances overall account security.

FUNCTIONS:

5. Strong Password Generation Function

- The **generate_strong_password** function creates a secure password of a specified length (default is 16 characters).
- It combines letters, digits, and punctuation symbols to ensure a strong password.
- The function uses the secrets module to randomly select characters, enhancing security against predictable password generation.
- If an error occurs during password generation, it prints the error message and raises the exception for further handling.

6. AES Password Encryption Function

- The **encrypt_password_aes** function encrypts a given password using the AES-GCM encryption algorithm.
- It generates a 12-byte nonce for encryption, then encrypts the password with the provided AES key and the generated nonce.
- The function returns the encrypted password, encoded in base64 format, which combines the nonce and the ciphertext for safe storage.
- If an error occurs during encryption, it catches the exception, prints an error message, and raises the exception again.

7. AES Password Decryption Function

- The **decrypt_password_aes** function decrypts an AES-encrypted password using a specified AES key.
 - It first decodes the base64-encoded encrypted password and separates the nonce (initialization vector) from the encrypted data.
 - Using the AESGCM cipher, it decrypts the password and returns the plaintext version.
- If any error occurs during the decryption process, it prints an error message and raises an exception.

8. Shared Key Derivation Function

- The **derive_shared_key** function generates a shared key using the given private key and a peer's public key through **Elliptic Curve DiffieHellman (ECDH)** key exchange.

- It then creates a random salt and uses the **PBKDF2HMAC** algorithm with SHA-256 to derive a secure key from the shared key, applying 100,000 iterations for added security.
- If any errors occur during this process, they are caught and logged.
- The function ultimately returns the derived secure key for further use.

9. ECC Key Generation Function

- The ``generate_ecc_keys`` function generates a pair of elliptic curve cryptography (ECC) keys using the **SECP256R1** curve, creating a private key and deriving the corresponding public key.
- If the key generation is successful, both keys are returned; if there's an error, it prints an error message and raises an exception.
- This function is crucial for secure cryptographic key management.

10. Email Existence Check Function

- The ``check_email_exists`` function checks if a given email address already exists in the database.
- It executes a SQL query to select the email from the ``user`` table where it matches the provided email.
- The result is stored in ``existing_user``, and the function returns ``True`` if the email exists and ``False`` otherwise.
- This function is useful for validating user registration or login processes.

11. Password Validation Function

- The ``validate_password`` function checks the strength of a given password based on specific criteria.
It verifies that the password is at least 8 characters long, contains at least one uppercase letter, and includes at least one special character from a predefined set.
- If any of these conditions are not met, it returns ``False`` along with an appropriate error message.
- If the password meets all requirements, it returns ``True`` with no error message.

12. Average Password Strength Calculation

- The **calculate_average_strength** function computes the average strength of a list of passwords.
- It initializes a total strength counter and iterates through each password to check if it meets the defined policy; if it does, the strength is set to 0.
- If not, a basic scoring mechanism evaluates the password's strength based on its length, capped at 100.
- Finally, the function returns the average strength as a percentage, logging any errors encountered during the evaluation.

13. Password Strength Analysis Function

- The **analyze_password_strength** function assesses the strength of a given password based on its length.
- It first checks if the input is a string, logging a warning if it's not, and returns 'Unknown.'
- Then, it categorizes the password as 'Weak' if it's less than 8 characters, 'Moderate' if it's between 8 and 11 characters, and 'Strong' if it's 12 characters or more.
- This function provides a simple method to evaluate password security.

14. Password Strength Distribution Analysis

- The **get_password_strength_distribution** function analyzes password strengths from a given DataFrame.
- It applies the **analyze_password_strength** function to the 'password' column, creating a new column for password strengths.
- The function then counts the occurrences of each strength level and returns the results as a dictionary.
- This allows for easy access to the distribution of password strengths across the dataset.

15. Old Item Deletion Function

- The **delete_old_items** function connects to the database to remove records from the **delete_items** table that are older than 30 days.
- It calculates the date from thirty days ago and executes a DELETE query using that date as a filter.
- If an error occurs during the operation, it prints the error message and rolls back the transaction to maintain data integrity.
- Finally, it ensures the database cursor and connection are closed to free up resources.

16. Custom Jinja2 Base64 Encoding Filter

- The **b64encode_filter** function is defined as a custom Jinja2 template filter that encodes data using Base64 encoding.
- It checks if the input data is not empty, then encodes it and decodes the result to a UTF-8 string.
- If the input is empty, it returns an empty string.
- This filter is then registered with the Jinja2 environment so it can be used in templates to easily encode data for various purposes.

17. One-Time Password (OTP) Email Send

- The **send_otp** function generates a random 6-digit OTP and sends it to a specified email address using the SMTP protocol.
- It stores the OTP in the session for later verification and constructs an email message containing the OTP.
- The function attempts to log in to the SMTP server and send the email, returning a success or error message based on the outcome.

FLASK APPLICATION ROUTE MANAGEMENT

18. User Login Route Implementation

- The login function handles both GET and POST requests for the login page.
- On a POST request, it retrieves the user's email and password from the form, then executes a SQL query to check if the credentials are valid while ensuring input sanitization to prevent SQL injection.

- If the credentials are correct, it stores the user's email in the session and redirects to the main page; otherwise, it renders the login page with an error message.
- For GET requests, it simply displays the login page.

19. Password Generation and Encryption Endpoint

- The **generate_password** function is a Flask route that generates a 16character strong password and encrypts it using derived AES keys from ECC key pairs.
- It uses ECDH to establish shared keys for encryption and returns both the generated and encrypted passwords as a JSON response.
- Any errors during the process are caught and returned with a 500-status code.

20. User Signup Functionality

- The signup function handles user registration, processing both GET and POST requests.
- It validates the input, checks for existing email addresses, hashes the password, and encrypts it using AES before storing user information in the database.
- If successful, it redirects the user to the login page; otherwise, it renders an error message on the signup page.

21. Password Recovery Route

- The **forgot_password** function handles password recovery by processing both GET and POST requests.
- When a POST request is made, it checks if the provided email exists in the database and, if so, generates and sends an OTP to that email while storing it in the session for verification.
- If the email is not registered, it returns an error message to the user on the forgot password page.

22. OTP Verification Endpoint

- The **otp_verification** function handles OTP verification by checking if the user's email is in the session; if not, it redirects them to the login page.
- Upon receiving a POST request, it compares the user-inputted OTP with the stored OTP in the session, providing feedback for invalid entries and redirecting to the reset password page upon successful verification.

- If the request method is GET, it simply renders the OTP verification form.

23. Password Reset Route

- The **reset_password** function handles the password reset process for users.
- It checks if the user's email is stored in the session; if not, it redirects to the login page.
- When a POST request is made, it updates the user's password in the database and clears the session before redirecting back to the login page.

24. User Logout and Session Clearing

- The **logout** function clears the user session by removing the `user_id` and `username` from the session.
- After clearing the session, it redirects the user to the login page using **redirect(url_for('login'))**.
- This ensures the user is logged out securely.

25. User Data Fetch and Profile Rendering

- This route checks if the user's email is stored in the session, and if not, redirects them to the login page.
- It then fetches the user's name and profile image from the database using the email and stores this information in the session.
- Finally, it renders the `db1.html` template, passing the user's name and profile image.

26. Calculate Overall Password Strength for User

- This route, **/calculate_overall_strength**, fetches passwords for the logged-in user from the database, calculates their average strength, and returns the result as JSON.
- If the user is not logged in or an error occurs during the process, it returns an error message.
- The function ensures secure handling of user data and password strength calculation.

27. Profile Update Function

- This function handles updating a user's profile.

- It checks if the user is logged in, retrieves and updates profile information like mobile, address, and profile image in the database, and updates the session.
- If the request method is GET, it fetches existing user data to display on the profile page.

28. Dashboard Route for Password and Folder Overview

- This **dashboard** route checks if the user is logged in by verifying the session, then retrieves and counts the user's passwords and folders from the database, along with the strength categories (**strong**, **moderate**, **weak**) of the passwords.
- These values are passed to the dashboard.html template for display. • If any error occurs, it logs the error and returns a 500 response.

29. Dashboard Data Retrieval and Password Strength Analysis

- This route **/dashboard/data** retrieves the logged-in user's passwords, processes them to clean invalid entries, and analyzes password strength.
- It returns the strength distribution as labels, values, and colors for chart representation.
- If any error occurs, an appropriate error message is returned.

30. User Passwords Page Route

- This route **/passwords** checks if the user is logged in and retrieves the user's email from the session.
- It then fetches the user's passwords and folder names from the database and passes them to the passwords.html template for rendering. • If the user is not logged in, it redirects them to the login page.

31. User-Specific Password View Route

- This route checks if a user is logged in by verifying their email in the session, then fetches and displays weak passwords and folder names for the logged-in user from the database.
- If no data is found, it defaults to empty lists.
- Finally, the data is passed to the **view.html** template for rendering.

32. Retrieve and Display Password Entry Route

- The **/pwd/<name>** route checks if the user is logged in by verifying the session.

- It retrieves the password entry for the logged-in user based on the provided name and displays it in a form.
- If no password is found, it redirects to the passwords page.

33. Folder Form Route with User Authentication

- This route handles requests to display the contents of a folder, ensuring the user is logged in.
- It fetches passwords from the database based on the folder name and the logged-in user's email, then renders the **folder_form.html** template with the passwords.
- If the user is not logged in, they are redirected to the login page.

34. Add Site and Windows Functionality

- The **site and windows** route handles both GET and POST requests for adding a site to the password manager.
- It checks if the user is logged in and, if so, processes form submissions to insert new site details into the database, including password strength analysis.
- If the operation is successful, it redirects to the main page; otherwise, it renders the site form with available categories and folders.

35. Payment Processing Route

- The **payment** route handles both GET and POST requests for adding payment information.
- It checks if the user is logged in via the session, processes form data when submitted, analyzes the password strength, and inserts the details into the payments and password tables in the database.
- If successful, it redirects the user to the main page; otherwise, it renders the payment form with available categories and folders.

36. Edit Password Entry Route

- The **edit_password** route handles both GET and POST requests for editing password entries based on the user's session.
- If a POST request is made, it updates the password details in the database only if there are changes compared to the existing data; otherwise, it informs the user that no changes were detected.
- For a GET request, it retrieves the current password data to pre-fill the edit form and renders it.

37. CSV Password Import Function

- The **import_passwords** route allows logged-in users to upload a CSV file containing passwords.
- It validates the file, reads its content, checks for sufficient data, analyzes password strength, and inserts the information into the database.
- If any errors occur, the user receives feedback, and categories and folders are loaded for the import form.

38. User Settings Route Implementation

- The **settings** route checks if a user is logged in by verifying if their email exists in the session; if not, it redirects them to the login page.
- It retrieves user data from the database using the logged-in user's email and prepares a user dictionary to pass to the settings.html template.
- Finally, the user information is rendered on the settings page, displaying their profile details.

39. Password Category Count Retrieval

- The **category** route handles GET requests to fetch the count of passwords in different categories: 'site', 'payment', and 'windows'.
- It executes SQL queries to retrieve the count for each category and assigns default values of 0 if no results are found.
- Finally, it renders the **category.html** template, passing the counts for each category as variables.

40. Retrieve Payment Passwords by Category

- The **category_form_payment** function is a Flask route that retrieves payment-related passwords for the logged-in user based on the specified category.
- If the user is not authenticated, they are redirected to the login page.
- The function fetches the relevant data from the database and renders the **category_form_payment.html** template with the passwords and headings.

41. Fetch Passwords by Category

- The **category_form** route checks if a user is logged in and redirects to the login page if not.
- It retrieves the user's email from the session and fetches passwords for the specified category related to that user from the database.

- The function then renders the `category_form.html` template with the relevant category name, headings, and passwords.

42. Folder Management Routes

- The `/folders` route handles both GET and POST requests for displaying and adding folders, while the `/rename_folder` route allows users to rename an existing folder.
- Both routes ensure that the user is logged in and interact with a database to manage folder data associated with the user's email.

43. Trash Route for User Deleted Records

- The `/trash` route in the Flask application checks if the user is logged in, redirecting to the login page if not.
- It deletes records older than 30 days from the `delete_items` table for the current user and retrieves the deleted items to display in the `trash.html` template.
- The deleted items include details such as name, item type, and deletion timestamps.

44. Routes for Deleting Folders and Passwords

- The `delete_folder` function deletes a specified folder along with its associated passwords, logging the details into a `delete_items` table.
- The `delete_password` function removes a specific password while ensuring it belongs to the logged-in user, also logging the deletion.

45. Routes for Trash Management

- The `restore_all` and `restore_selected` routes restore deleted items back to their original tables based on user input, while `delete_selected` and `delete_all` routes remove items permanently from the trash.
- Each function checks if the user is logged in and handles errors gracefully, providing appropriate feedback through flash messages.

JAVASCRIPT FUNCTION

1. Loading Content & Active Link Handling:

- `loadContent(url)` fetches content from a given URL and updates the DOM with the result.
- `setActiveLink(clickedLink)` sets the active state for a sidebar link.

- **handleClick(url, clickedLink)** integrates both functionalities to load content and highlight the active link.

2. Form Handling:

- **handleSubmit(event)** processes a form submission via a POST request to /folders.
- **handleFormSubmit(event)** handles form submission for redirecting to the passwords page after submission.

3. File Download Functions:

- **downloadTableAsCSV()** downloads the content of a table as a CSV file.
- **downloadTableAsPDF()** uses jsPDF to convert a table into a PDF format.

4. Popup & Modal Functions:

- **openPopup(name)** and **closePopup()** handle showing and hiding a popup.
- **showModal()** and **hideModal()** handle displaying modals.
- **togglePopup()** toggles the visibility of a popup element.

5. Password Strength:

- **fetchPasswordStrength()** fetches and updates password strength data for a circular progress indicator.

6. Handling Dashboard Clicks & Chart Rendering:

- **handleClickDashboard(route, element)** fetches content for the dashboard and updates the UI.
- **fetchChartData()** fetches chart data for rendering a polar area chart using Chart.js.

7. Restore & Delete Items:

- **restoreAll(url)** and **deleteAll(url)** handle restoring and deleting all items.
- **handleSelectedItems(actionUrl, isRestore)** processes selected items for restoration or deletion.

8. Password Management:

- **deletePassword(url, rowElement)** handles password deletion with confirmation.

9. Checkbox & Button State:

- **updateButtonText()** updates button text based on checkbox selection for restore/delete actions.

TEMPLATES:

1. Login Page Design

- This HTML code creates a split-screen login page for a password manager application.
- The left section includes a form for users to log in, with email and password input fields, a "Forgot Password" link, and buttons for social media login options.
- The right section contains an image with a background gradient.
- CSS styling ensures a clean, responsive layout with fixed footer text.

2. Sign-Up Page with Password Strength Checker

- This HTML code creates a responsive sign-up form with two split sections: one for the form and another for an image.
- It includes password strength validation, where users can see real-time feedback on password requirements (length, uppercase, lowercase, numbers, and special characters) using a progress bar.
- A button allows users to generate a strong password automatically, with visual feedback on password strength shown in real-time.

3. Forgot Password Page Layout

- This HTML template is for a "Forgot Password" page within a password manager application.
- It features a split screen layout with a form on the left side for users to input their email to recover their password, and an image on the right.
- The design includes some basic styling for the form, buttons, and layout, ensuring responsive and centered elements.
- The page also displays a logo and copyright notice at the bottom.

4. OTP Verification Page Layout

- This HTML template displays a split-screen layout for an OTP (One-Time Password) verification page in a password manager.
- The left section contains a form where users enter a 6-digit OTP received via email, while the right section displays an image.
- The form is centered and styled with a shadow, with error messages shown if verification fails.
- The layout is responsive, with copyright information fixed at the bottom.

5. Password Reset Page

- This HTML page creates a two-column layout for a password reset feature in a password manager.

- The left side includes a form for resetting the password, with inputs for the new and confirm password fields, while the right side features an image.
- The layout uses CSS for styling, including centering elements and adding responsive features like flexible width and button positioning.

6. Sidebar Navigation

- This HTML page features a dashboard with a sidebar navigation that includes options like Dashboard, Passwords, Folders, and Settings.
- The layout incorporates a search bar, a logo, and a profile dropdown button.
- The sidebar stays fixed while the content dynamically updates based on user actions, and the styling uses external libraries like FontAwesome and jsPDF for icons and PDF generation.

7. Dashboard

- This HTML page features a dashboard that displays the count of passwords, folders, and weak passwords in a card format.
- A Chart.js polar area chart visualizes password strength levels (strong, moderate, weak) with dynamic updates based on user data.
- Fetch functions retrieve and display password statistics, while the UI includes cards showing the total count of each category.

8.Password page

- This code implements a responsive webpage layout with a dropdown menu and table for displaying data.
- It features JavaScript functions for handling form submissions, content loading, and popup interactions for adding or deleting items.
- The table includes sorting functionality and scrollbars for better user experience.

9. Adding a Site

- This HTML document creates a responsive form for users to add a new site with fields for name, category, folder, username, web address, and password.
- It includes styles for layout and design, utilizing a gradient background and custom button effects.
- The password input features a toggle icon to show or hide the password.
- Additionally, there's a back button that redirects to the passwords page and JavaScript for content loading via fetch.

10. Payment Form

- This HTML document defines a payment form with a responsive layout, including fields for name, category, folder, username, account number, IFSC code, bank name, branch name, web address, and password.
- The form features a toggle for showing/hiding the password input and utilizes a clean design with a gradient background, rounded edges, and interactive buttons.
- It also includes a back button to navigate to the passwords page.
- The form is styled with CSS and dynamically populated using Jinja templating.

11. Import Passwords

- The import.html template provides a user interface for importing passwords via a CSV file in a password manager application.
- It features a form where users can select a category, optionally choose a folder, and upload a CSV file, all styled with CSS for a modern look.
- The layout includes a back button for navigation and dynamically populates category and folder options based on the backend data.
- Upon form submission, the data is sent to the /import route for processing.

12. Username and Password Popup Form

- This HTML code creates a popup form for entering a username and password, featuring functionality for copying these values to the clipboard.
- It includes a password visibility toggle and displays confirmation messages upon copying.
- The form is styled for a clean user interface with responsive elements, ensuring an intuitive experience.
- The JavaScript functions handle the display logic for the popup and the clipboard interactions.

13. Update Password Form

- This HTML code represents a user interface for updating a password in a password manager application.
- It includes a form with fields for entering details such as name, category, folder, email, password, and website, along with a toggle feature to show or hide the password.
- The form is styled with CSS for a modern and clean look, utilizing a responsive layout to enhance user experience.

- It also includes a back button to navigate to the previous page.

14. Folders Page

- This HTML code represents a user interface for managing folders in a web application.
- It features a modal for adding new folders and another for renaming existing ones, both equipped with input fields and buttons for submission.
- The design incorporates responsive styling, hover effects, and dropdown options for improved user experience.
- Additionally, it includes a table for displaying folder details with alternating row colors and a sticky header for easy navigation.

15. Folder-Specific Passwords

- This HTML template is designed for displaying a list of passwords associated with a specific folder in a password manager application.
- It features a responsive table layout that includes columns for the password name, email/username, password itself, category, and a clickable URL link.
- The styling is managed through CSS to provide a clean and modern look, including hover effects and zebra striping for improved readability.
- Additionally, it includes a back button for navigation to the folders page.

16. Category Buttons Layout

- The provided HTML code creates a responsive web page displaying category buttons for "Site," "Payment," and "Windows," each represented by a card-like layout.
- Each button includes an icon, category name, and a count of items in that category, which is fetched from the server.
- The layout uses flexbox for alignment and spacing, and the buttons feature hover effects for a more interactive experience.
- JavaScript is used to handle click events and navigate to the respective category forms.

17. Categorized Password

- This HTML template displays a list of passwords categorized by the `category_name` variable, featuring a sortable table with password details and links.
- Users can navigate back to the previous category and download the table as CSV or PDF using the buttons provided.

- The design includes responsive styles, a fixed table header, and hover effects for better user experience.
- External libraries like jsPDF are included for PDF generation functionality.

18. Profile Update Page

- This HTML code creates a responsive form for updating an employee's profile, including options to upload a profile image and enter personal information.
- The layout features a centered form with a circular profile image placeholder, which changes upon file upload.
- Input fields for user name, email, mobile phone, and permanent address are included, with the email field set to read-only.
- The form is styled with a modern design and includes hover effects on the submit button for an enhanced user experience.

19. Settings Page

- This HTML code represents a user profile settings page featuring a responsive layout for various action buttons.
- The buttons include options like "Update Profile," "About Us," and "Change Master Password." Each button arrangement contains an icon and label, with styles applied for hover effects and a consistent design.
- The structure uses flexbox for a neat arrangement of buttons, ensuring a visually appealing and user-friendly interface.

20. About Us Page

- This HTML document outlines the "About Us" page for CyberVault Innovations and Technologies, emphasizing their commitment to secure password management solutions.
- It describes the use of advanced encryption standards like AES 256 and ECC algorithms to protect user passwords.
- The page features a clean design with centered text and a responsive layout for an optimal user experience.
- Overall, it highlights the company's dedication to innovation and user security in digital asset management.

21. Trash Page

- This HTML template creates a "Trash" page for a password management application.

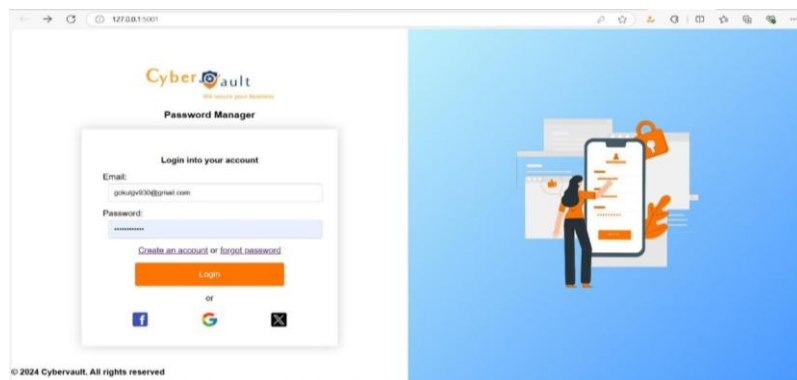
- It includes a styled table displaying a list of passwords or folders, with options to restore or delete selected items.
- The table features sorting functionality and checkbox inputs for multiselection.

GITHUB:

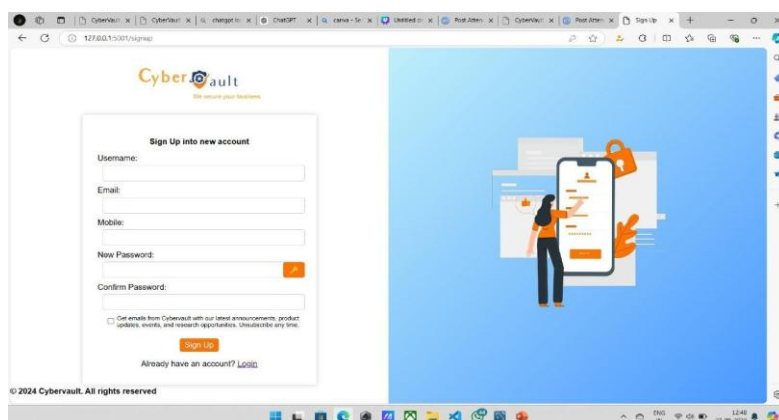
<https://github.com/Gokul345/Password-Manager.git>

UI Screenshot:

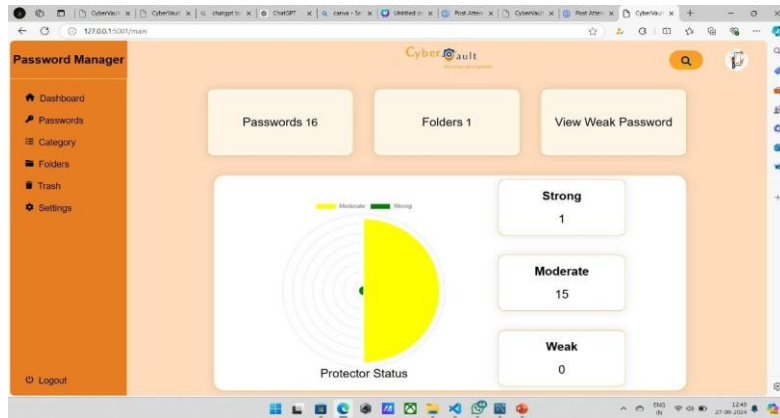
Login Page



Signup Page:



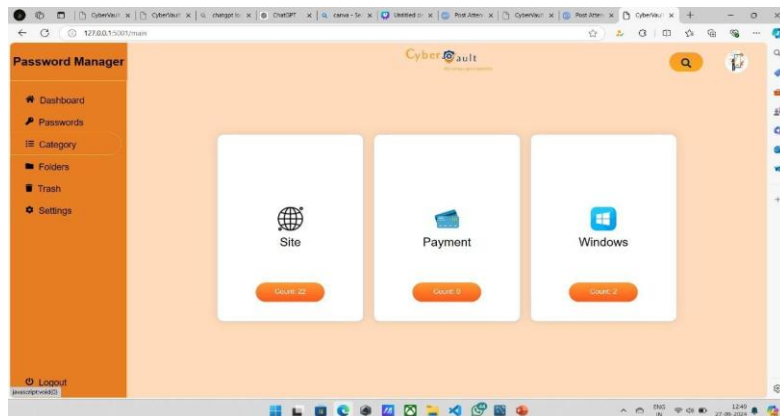
Dashboard.html



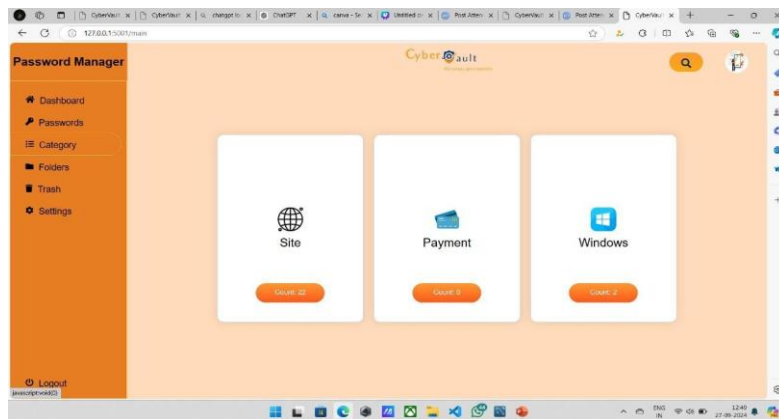
Password.html

Name	Category	Folder	Created_At	Created_Time	Status	URL	Action
accounts.google.com	Site	Site	2024-09-24	15:52:13	Moderate		
accounts.google.com	Site	Site	2024-09-24	15:52:13	Moderate		
app.cookiecutter.com	Site	Site	2024-09-24	15:52:13	Moderate		
edvanceon.in	Site	Site	2024-09-24	15:52:13	Moderate		
goldmansachs.faf.net	Site	Site	2024-09-24	15:52:13	Strong		
http://127.0.0.1:5000	Site	Site	2024-09-24	15:52:13	Moderate		
http://127.0.0.1:5001	Site	Site	2024-09-24	15:52:13	Moderate		
http://127.0.0.1:5090	Site	Site	2024-09-24	15:52:13	Moderate		
http://127.0.0.1:5090	Site	Site	2024-09-24	15:52:13	Moderate		
learning.tokenhub.in	Site	Site	2024-09-24	15:52:13	Moderate		
login.live.com	Site	Site	2024-09-24	15:52:13	Moderate		

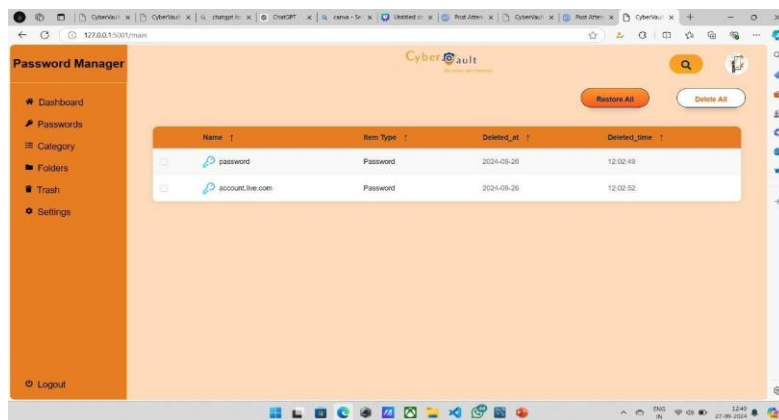
Category.html



Folder.html



Trash.html



Settings.html

